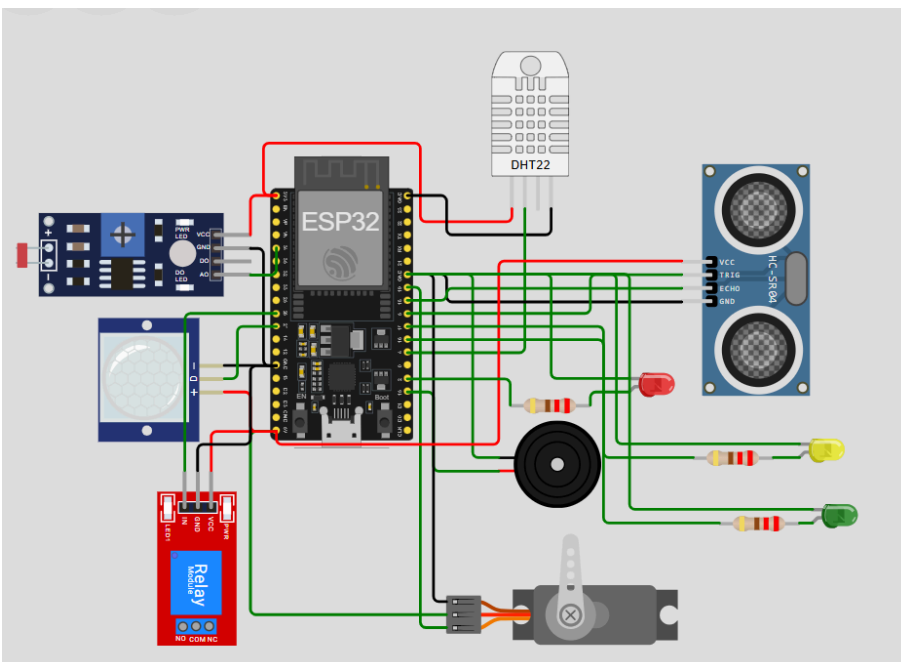
Assignment 1

Team: **Elhusseain Aboulfetouh 202202239, Kareem Adel 202200059**

1. System Description

This project implements a **smart environment monitoring and control system** using an **ESP32 microcontroller** in the **Wokwi simulation environment**, multiple virtual sensors, and **Node-RED** for visualization and control. The system collects environmental data including **temperature, humidity, light level, motion detection, and distance** using simulated sensors such as the **DHT22, LDR, PIR sensor, and ultrasonic sensor**.

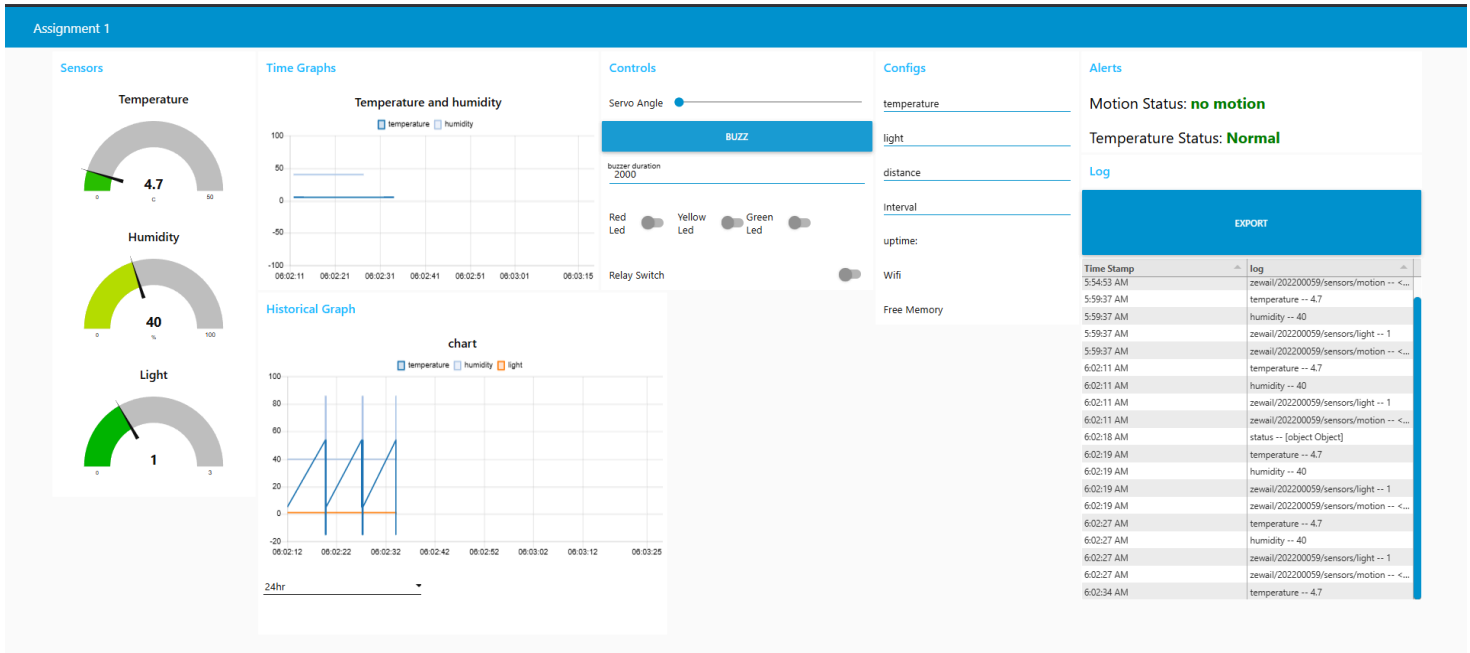
The ESP32 reads sensor values and publishes them to an **MQTT broker (HiveMQ)**. Node-RED subscribes to these MQTT topics to visualize the data on a dashboard using gauges, charts, and status indicators. The system also controls actuators including LEDs, a buzzer, a relay module, and a servo motor, all simulated in Wokwi.



The system supports both **automatic and manual control**:

- **Automatic control:** implemented using configurable thresholds for temperature, light, and distance. For example, when the temperature exceeds a defined threshold, the relay and a red LED are activated. When light levels drop below a threshold, a yellow LED turns on. Motion detection triggers the buzzer, and object proximity controls the servo position.

- **Manual control:** Node-RED allows manual control of actuators through dashboard switches and sliders.



Additional features include logging of sensor readings and actuator events, as well as monitoring features such as heartbeat messages that report uptime, simulated WiFi signal strength, and available memory.

---

2. Challenges Faced

Even in a simulated environment, we encountered several technical challenges:

1. **Simulated Sensor Behavior:**

    - Some virtual sensors, like the DHT22, occasionally returned inconsistent or NaN values when polled too quickly. This required adding delays and error handling in code.
    - The LDR sensor and ultrasonic sensor readings in **Wokwi** needed mapping and threshold tuning to simulate realistic responses for automatic control.

2. **Buzzer Timing and Control:**

- Implementing the buzzer using `digitalWrite` caused delays and blocked the main loop, affecting servo and LED control.
- Switching to the **`tone()`** function resolved this, allowing non-blocking buzzer control for both manual and motion-triggered events.

3. **MQTT Communication:**

   - Maintaining a stable MQTT connection even in simulation required implementing reconnection logic to handle Wokwi network simulation delays.
   - Parsing incoming MQTT messages from Node-RED sometimes failed due to incorrect JSON formatting, requiring robust string handling.

4. **Servo and Actuator Conflicts:**

   - Automatic servo movement based on distance sometimes conflicted with manual dashboard control. State flags were needed to differentiate manual vs automatic control.
   - Ensuring multiple actuators (LEDs, relay, servo, buzzer) operated concurrently without blocking the loop required non-blocking, event-driven code.

5. **Node-RED Dashboard Design:**

   - Designing a dashboard that presents sensor readings, allows manual control, and provides system monitoring was iterative.

   - Ensuring that threshold adjustments and control messages from the dashboard correctly reflected in the ESP32 simulation required careful testing and debugging with Wokwi outputs.

6. **Simulation Limitations:**

   - Some behaviors in Wokwi, such as motion detection or analog sensor readings, were simplified or idealized compared to real hardware.

   - Timing and response of actuators had to be adjusted to match the simulation environment, which sometimes differed from real-world behavior.

---

3. Lessons Learned

Working in a simulation environment provided practical experience in building IoT systems without real hardware. Key lessons include:

- Learning to use MQTT as a lightweight protocol for real-time communication between simulated devices and Node-RED.
- Understanding non-blocking programming techniques on ESP32 to handle multiple sensors and actuators simultaneously.
- Gaining experience in event-driven control logic, state management, and threshold-based automation.
- Appreciating the role of visual dashboards (Node-RED) in monitoring, controlling, and testing IoT systems.
- Recognizing differences between simulation and real-world deployment, highlighting the need to validate systems with actual hardware later.

Overall, this project enhanced our understanding of IoT system architecture, sensor integration, actuator control, and real-time automation, while demonstrating the usefulness of simulation tools like Wokwi for rapid prototyping and testing.